Web-Based Visualization Part 1

Understanding the Browser with the Citi Bike Story

HTML · CSS · SVG

Citi Bike: design question and data

Dataset: April 2020 · 9,268 trips · 15 fields · COVID-19 lockdown

Design question: How did people's everyday lives change during lockdown?

Use Citi Bike trip patterns as one window into everyday life during lockdown.

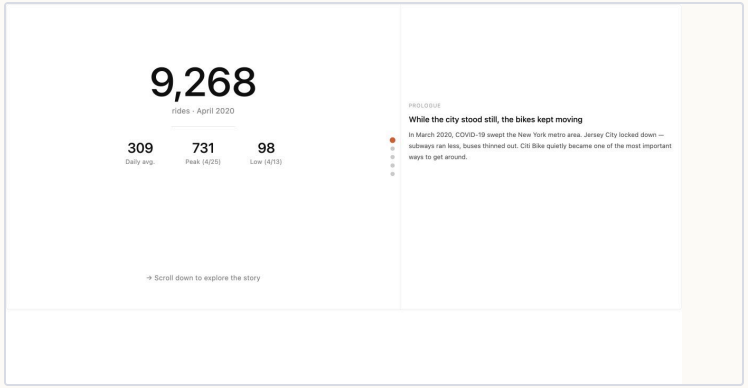
FIRST 4 RECORDS · 5 OF 15 SOURCE FIELDS

starttime	tripduration seconds	start station name	end station name	usertype
2020-04-01 01:06:20.6300	1437	Fairmount Ave	Fairmount Ave	Customer
2020-04-01 05:02:42.0570	264	Oakland Ave	Sip Ave	Subscriber
2020-04-01 06:20:28.1190	254	Fairmount Ave	Sip Ave	Subscriber
2020-04-01 06:33:30.5170	429	Sip Ave	Bergen Ave	Subscriber

What can we ask using these fields?

- When do trips occur?
- Where do they start or end?
- How does use differ by user type?

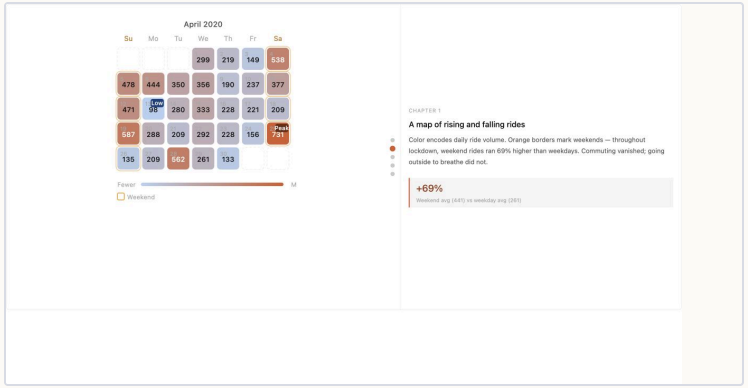
Citi Bike: riding through April 2020



Overview

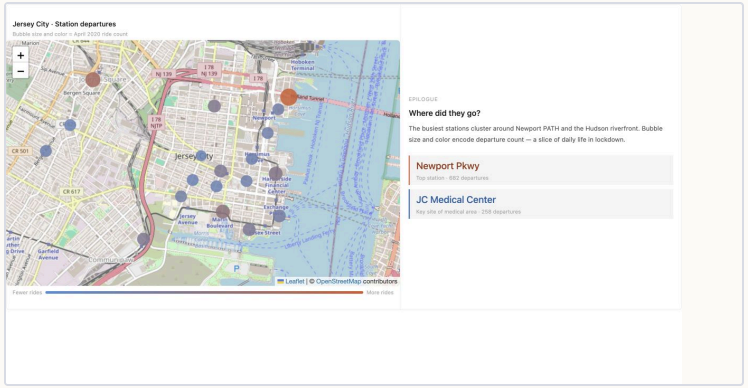
Establish the total and period.

Open the interactive Citi Bike story ↗



Daily calendar

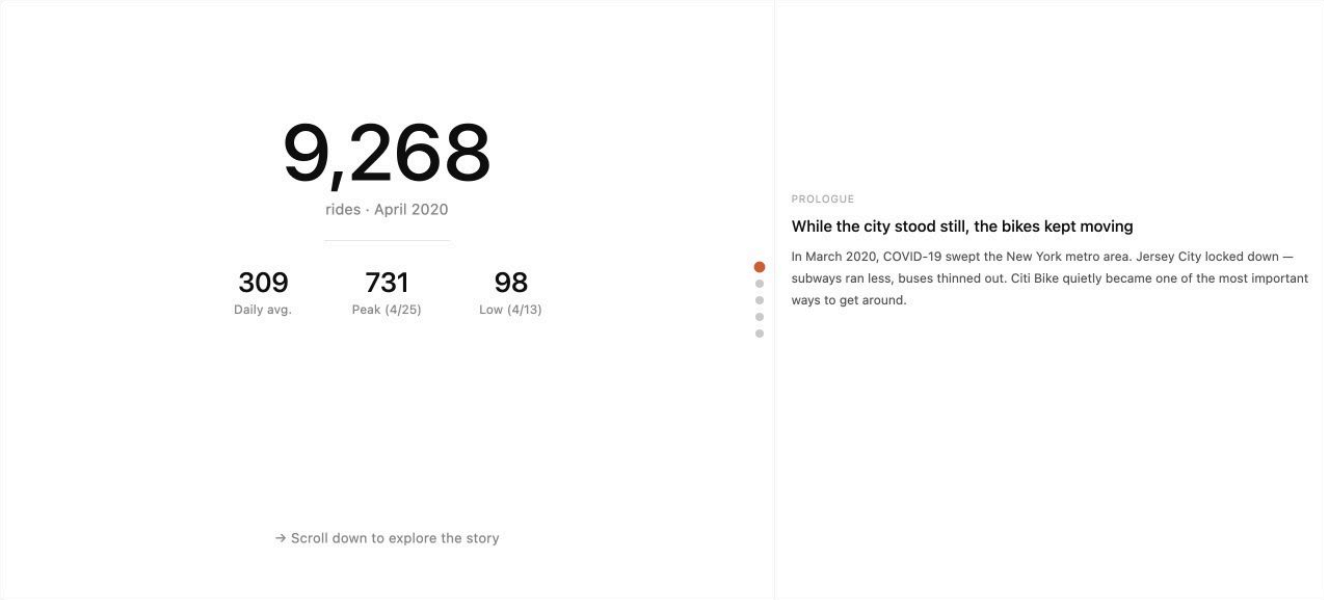
Compare dates and day types.



Station departures

Locate activity across the city.

1 · Context: establish the scope of the story



Opening design choices

Show: 9,268 rides and April 2020 as the starting point.

Annotate: introduce Jersey City and the lockdown setting in the narrative.

Keep: this geographic and temporal scope across later chapters.

Control: invite readers to scroll; use chapter dots to navigate the story.

2 · Questions: unpack the design question

Design question: How did people’s everyday lives change during lockdown?

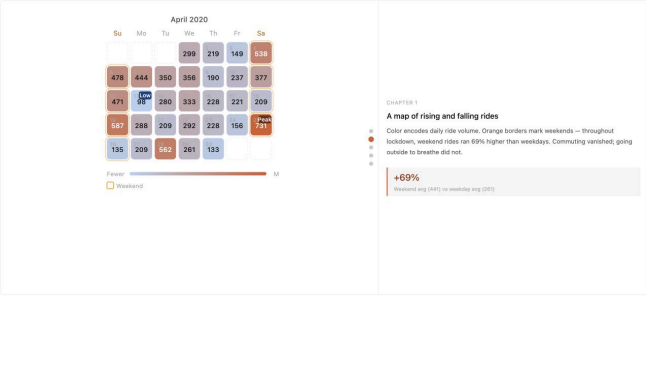
Data questions about Citi Bike use

- How did trip counts vary from day to day?
- How did average daily counts compare between weekdays and weekends?
- At what times of day did trips start on weekdays and weekends?
- How did trip shares and average durations differ by recorded user type?
- Which stations had the most departures, and where were they located?

3 · Evidence: match each question to a view

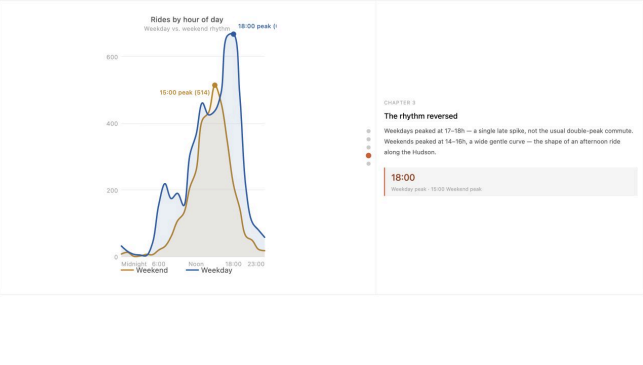
Q1–2 · Calendar heatmap + daily averages

Daily counts and weekday/weekend differences.



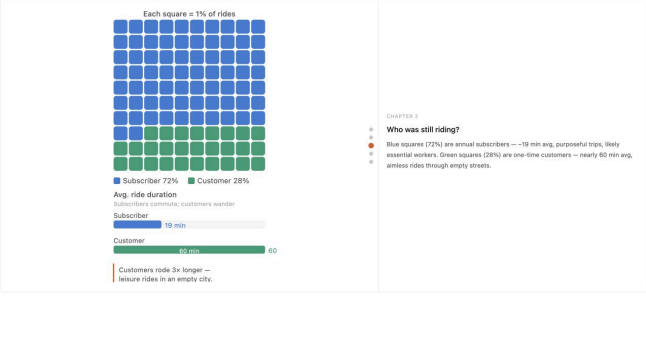
Q3 · Hourly line chart

When do trips start on weekdays and weekends?



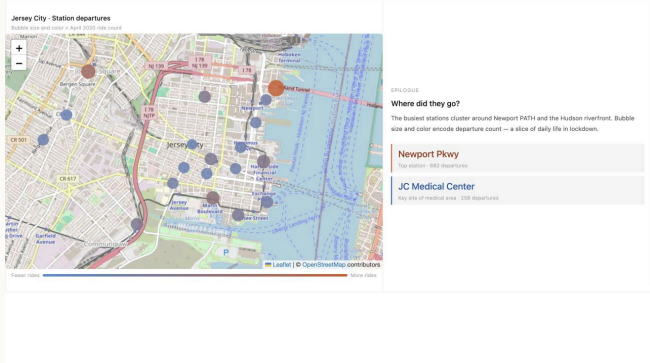
Q4 · Waffle chart + duration bars

Trip shares and average durations by user type.

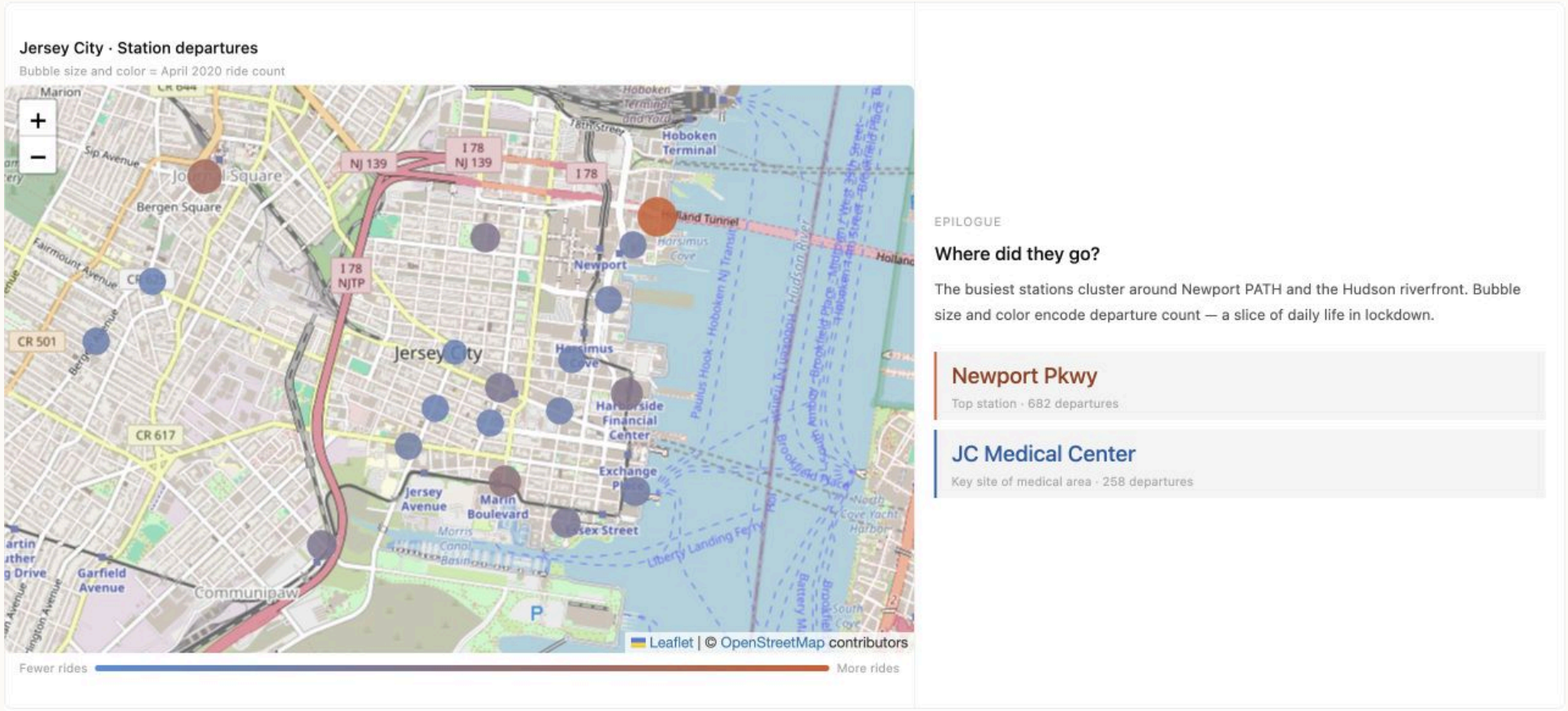


Q5 · Station departure map

Where are departures concentrated?



4 · Implication: connect bike use to everyday life



To implement this complete visualization design, Vega-Lite alone is not enough. We need to understand more **web-based visualization technologies**.

Agenda: build the visible page

01	**HTML**	Organize the content
02	**CSS**	Style and arrange the page
03	**SVG**	Draw the visualization

We will build the page one small step at a time.

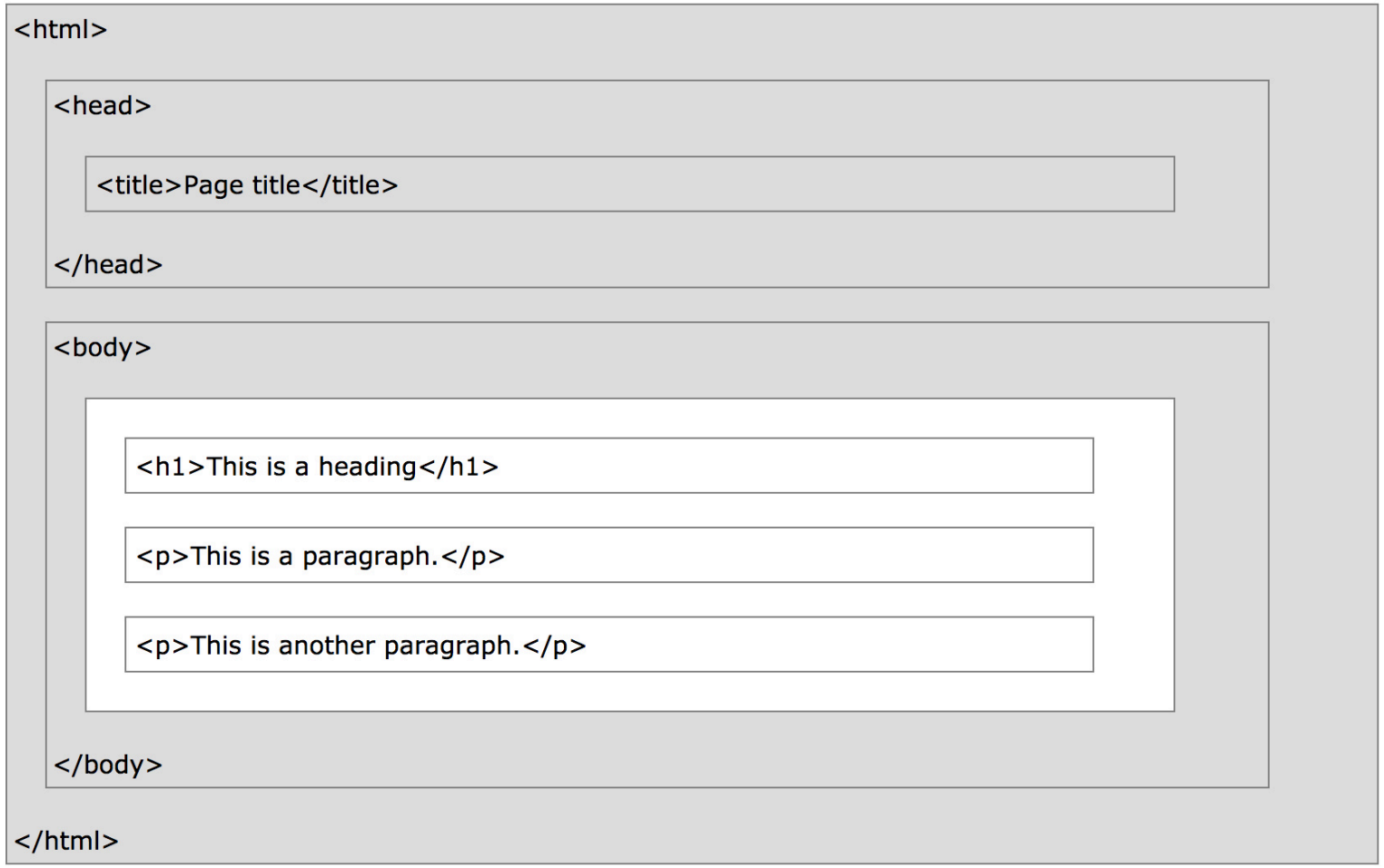
HTML gives content a role

HTML (HyperText Markup Language) is a markup language that gives web content its structure and meaning.

`head` contains information about the page.

`body` contains headings, paragraphs, images, and other visible content.

MDN · HTML basics



A complete HTML page for the Citi Bike story

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Citi Bike</title>
</head>
<body>
  <h1>Citi Bike rides</h1>
  <p>Jersey City · April 2020</p>
  <img src="calendar.svg"
       alt="April daily rides">
</body>
</html>
```



HTML tag syntax

An **HTML tag** is a markup label enclosed in angle brackets. Tags define an **element** and mark where it begins or ends.

```
<tagname attribute="value">content</tagname>
```

- `<tagname …>`: opening tag
- `</tagname>`: closing tag
- **Attributes** add information in the opening tag.

Void elements, such as `<img>` and `<br>`, have no content or closing tag.

```
<p lang="en-US"
   style="color: red;">
  Paragraph in English
</p>
```

Paragraph in English

There are many HTML tags; please refer to: W3Schools HTML tag reference ↗

Common HTML tags

`h1` : main heading · `p` : paragraph
`ul` : unordered list · `li` : list item · `a` : hyperlink

HTML code

```
<h1>Citi Bike rides</h1>
<p>Jersey City · April 2020</p>
<ul>
  <li>Daily counts</li>
  <li>Popular stations</li>
</ul>
<a href="https://citibikenyc.com">
  Visit Citi Bike
</a>
```

Displayed result

Citi Bike rides

Jersey City · April 2020

- Daily counts
- Popular stations

Visit Citi Bike

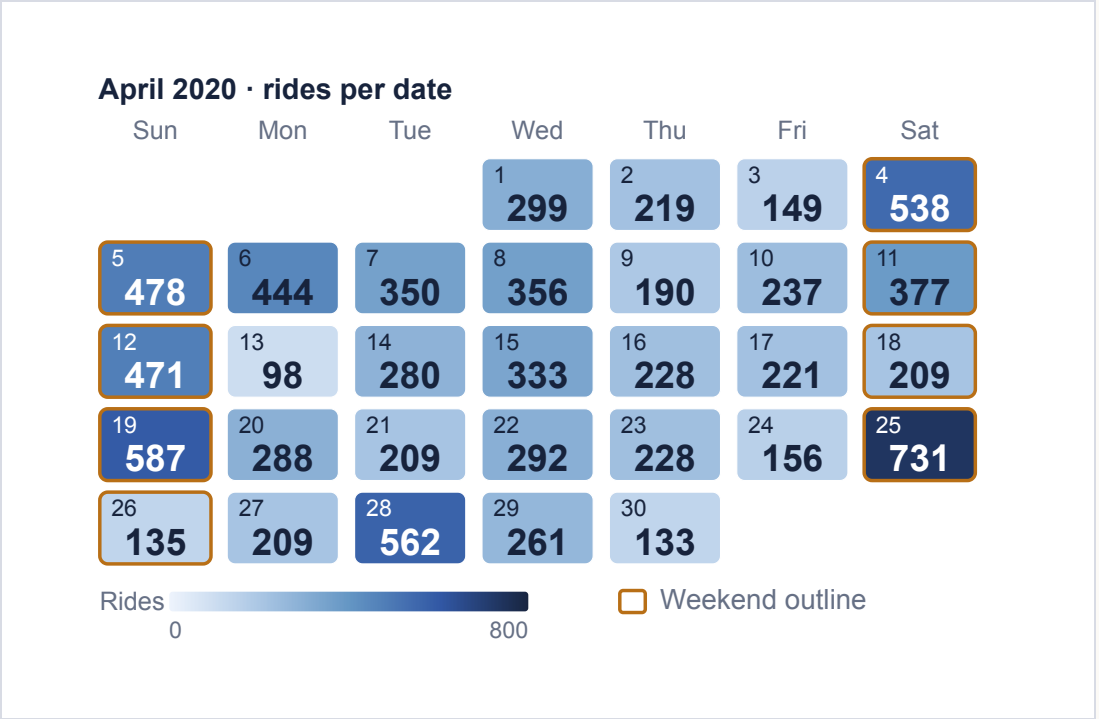
Reference: MDN · HTML elements

Place an image in the page

`img` embeds an image: `src` identifies its file, and `alt` supplies a text alternative.

```
<img src="calendar.svg"
  alt="April daily rides">
```

Keep `calendar.svg` beside the HTML file so this relative path resolves.



Name elements with id and class

An **id** identifies one element and must be unique within the document.
A **class** gives elements a shared group name.

HTML code

```
<section id="overview"
  class="story-section">
  <h2>Overview</h2>
</section>
<section id="calendar"
  class="story-section">
  <h2>Daily calendar</h2>
</section>
```

Displayed result

Overview

Daily calendar

Different ids:
`overview` and `calendar`
Shared class:
`story-section`

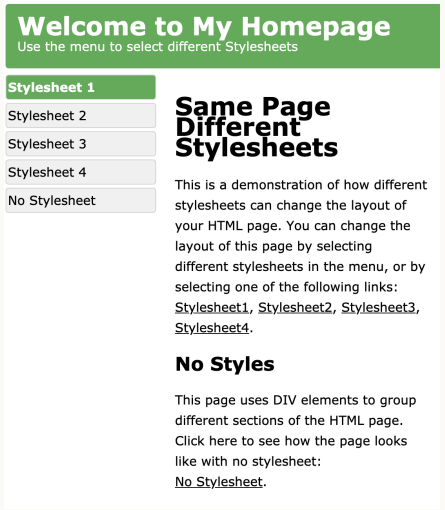
Reference: MDN · id / class

CSS controls how HTML is displayed

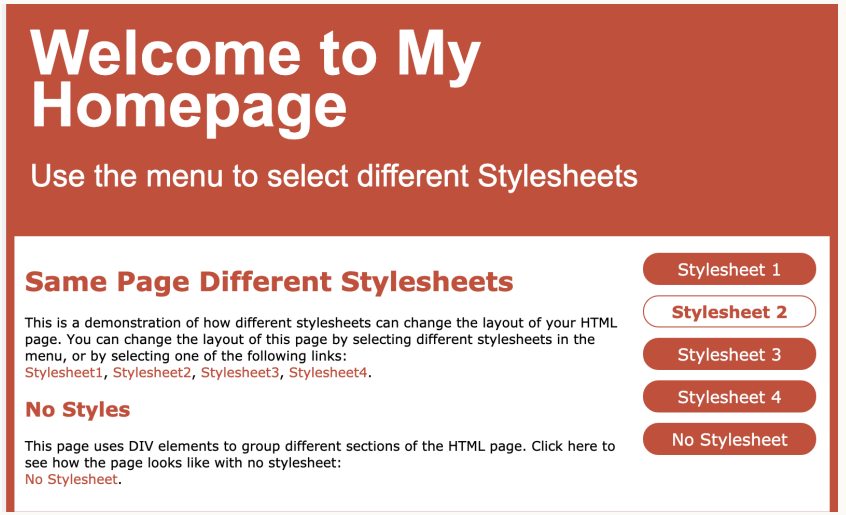
CSS (Cascading Style Sheets) is a style sheet language that controls the appearance and layout of HTML elements.

- **Selectors** choose which elements to style, including elements that share a class.
- **Style rules** set colors, fonts, spacing, borders, and layout.

Same HTML · Stylesheet 1



Same HTML · Stylesheet 2



W3Schools: Switch stylesheets ↗ · CSS introduction ↗

1. Internal CSS: use a style element

An **internal stylesheet** contains CSS rules inside a `style` element in the HTML `head`. The rules apply to matching elements in that document.

```
<head>
  <style>
    h1 { color: #3157a4; }
  </style>
</head>
<body>
  <h1>Citi Bike</h1>
</body>
```

Source: MDN · The style element

Browser result

Citi Bike

The HTML and CSS are stored in the same file.

Here, the rule makes the `h1` text blue.

2. External CSS: link a stylesheet file

An **external stylesheet** stores CSS rules in a separate file. A `link` element in the HTML `head` connects that file to the page.

index.html (inside the document)

```
<head>
  <link rel="stylesheet"
        href="style.css">
</head>
```

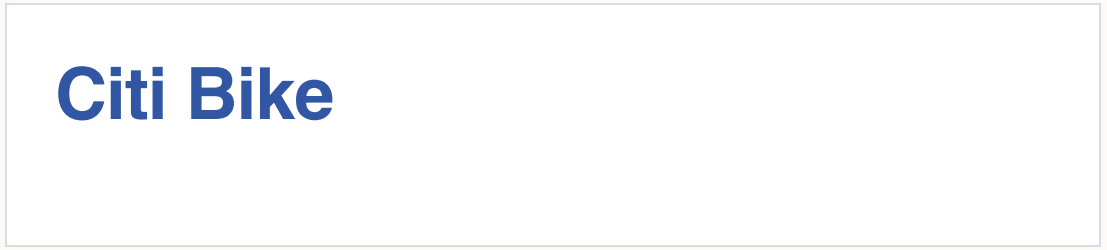
In this example, keep `index.html` and `style.css` in the same folder.

Several HTML pages can link to the same CSS file.

Source: MDN · The link element

style.css

```
h1 {
  color: #3157a4;
}
```



The body keeps the same `h1` element.

3. Inline CSS: use the style attribute

An **inline style** puts CSS declarations directly in an element's `style` attribute. These declarations apply to that element.

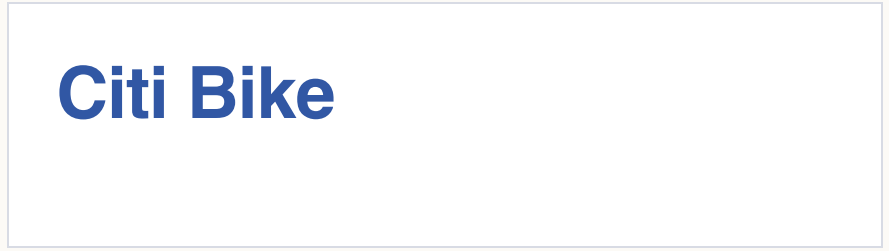
```
<h1 style="color: #3157a4;">
  Citi Bike
</h1>
```

Inside the quotes, write `property: value;`.

A colon separates the property and value; a semicolon separates declarations.

Source: MDN · The style attribute

The same browser result



The `style` attribute contains declarations, without a selector or braces.

A CSS rule has a selector and declarations

A **CSS style rule** consists of a selector and a declaration block. The selector matches elements; the declarations describe their styles.

General syntax

```
selector {
  property: value;
  property: value;
}
```

The braces `{ }` enclose the declaration block.

Source: MDN · CSS syntax

Reading a rule

- **Selector:** which elements to style.
- **Property:** which aspect to set, such as `color`.
- **Value:** the setting, such as `#3157a4`.

Each `property: value;` pair is a **declaration**.

A selector identifies which elements to style

A **CSS selector** is a pattern that matches elements in a document. Tag names, classes, and IDs provide three common ways to select them.

Selector	Syntax example	Matches
Tag (type)	`h2`	Every `h2` element
Class	`.story-section`	Every element with `class="story-section"`
ID	`#calendar`	The element with `id="calendar"`
Selector list	`h1, h2`	Elements matching either selector

The `.` and `#` belong to the CSS selector, not the HTML class or ID value.

Source: MDN · Basic selectors

A tag selector matches every element of that type

A **tag selector** uses an element's tag name. The selector `h2` matches both second-level headings below.

```
<h2>Overview</h2>
<h2>Daily calendar</h2>
```

```
h2 {
  color: #3157a4;
}
```

Source: MDN · Selectors

Both headings change

Overview

Daily calendar

A class selector styles a group

A **class selector** begins with `.` and matches elements that share that class name.

HTML

```
<section class="story-section">
  <h2>Overview</h2>
  <p>9,268 trip records</p>
</section>
<section class="story-section">
  <h2>Daily calendar</h2>
  <p>30 dates in April 2020</p>
</section>
```

CSS

```
.story-section {
  border: 2px solid #3157a4;
}
```

Source: MDN · Selectors

Both sections share one style

Overview

9,268 trip records

Daily calendar

30 dates in April 2020

An ID selector targets one element

An **ID selector** uses `#` to match an element by its unique `id`.

HTML

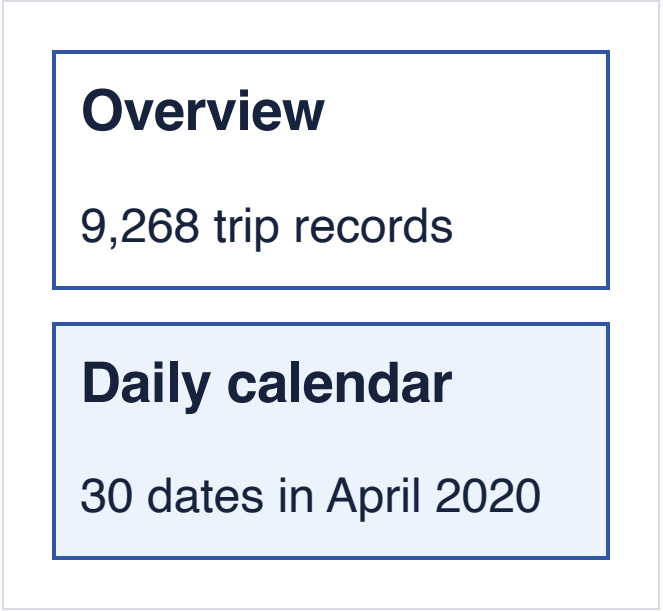
```
<section id="overview" class="story-section">
  <h2>Overview</h2>
  <p>9,268 trip records</p>
</section>
<section id="calendar" class="story-section">
  <h2>Daily calendar</h2>
  <p>30 dates in April 2020</p>
</section>
```

CSS

```
.story-section { border: 2px solid #3157a4; }
#calendar { background: #edf3fc; }
```

Source: MDN · Selectors

Only the calendar gets the fill



Make the text comfortable to read

`font-size` sets the size of text. `line-height` sets the height used for each line of text.

```
p {
  font-size: 26px;
  line-height: 1.5;
}
```

Each line uses

26 × 1.5 = 39px of height.

Source: MDN · font-size and line-height

Paragraph result

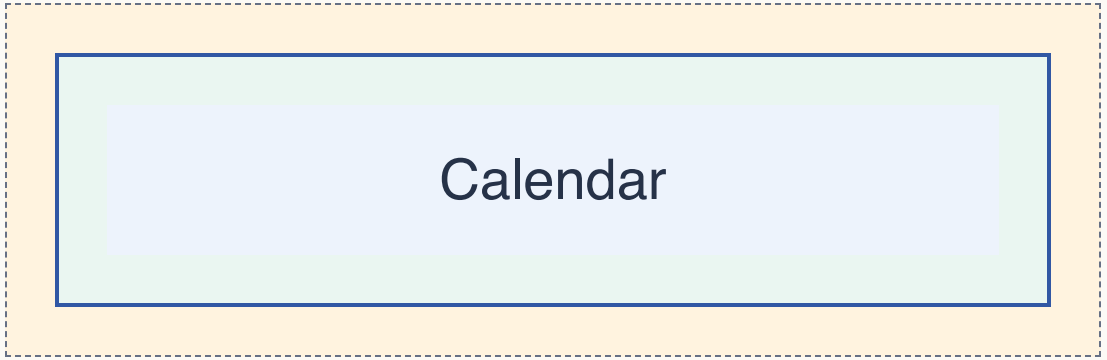
In April 2020, the Jersey City file contains 9,268 trip records across 30 dates.

Spacing lives inside and outside the border

Padding is space between content and its border. **Margin** is space outside that border.

```
.story-section {
  padding: 24px;
  margin: 24px;
}
```

One value applies to all four sides.



Padding: inside

Margin: outside

The blue line is the border.

Source: MDN · padding and margin

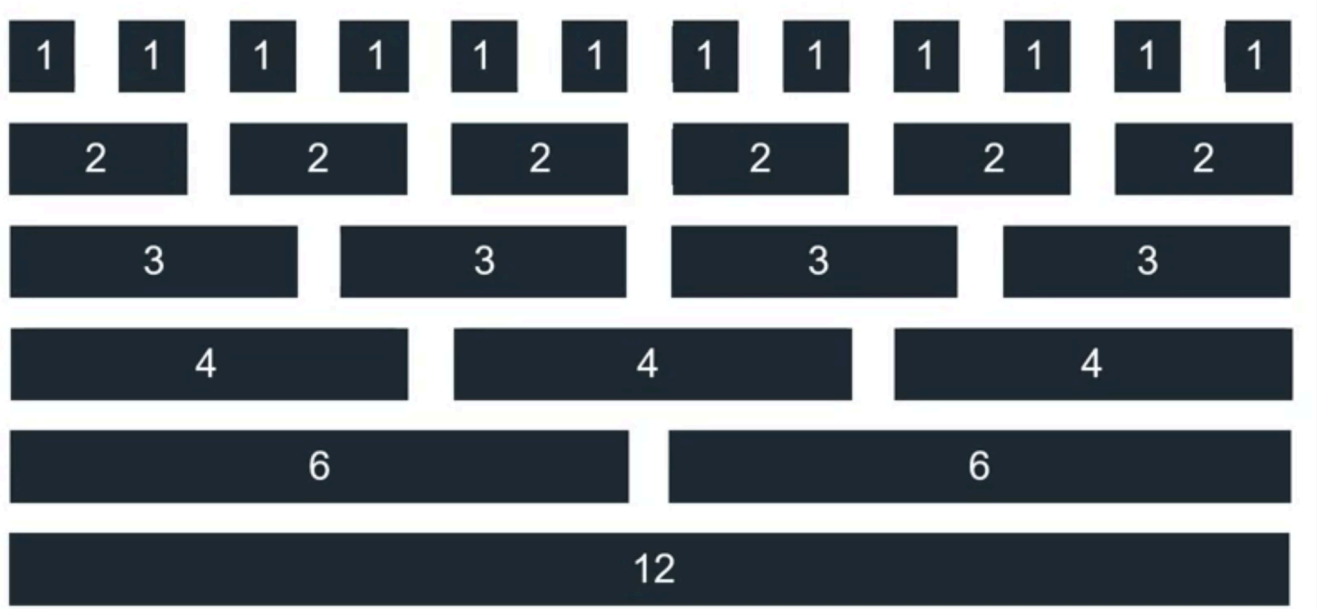
Bootstrap: reusable styles and a 12-column grid

Bootstrap is a front-end toolkit with reusable CSS styles and components for building web pages.

Its **grid system** divides a row into **12 equal-width units**. Each content column spans one or more units.

- **6 + 6:** two equal columns.
- **4 + 8:** a 1:2 column-width ratio.

Organize the page as `container` → `row` → columns.



For the grid, bootstrap uses a 12 column grid system that can be fluid or fixed.

Reference: Bootstrap · Grid system

Load Bootstrap, then assign layout classes

In `head`: load the stylesheet

```
<link rel="stylesheet" href="bootstrap.css">
```

In `body`: container → row → columns

```
<div class="container">
  <div class="row">
    <div class="col-md-4">
      <h2>Citi Bike</h2>
      <p>April 2020: 9,268 trips</p>
    </div>
    <div class="col-md-8">
      <img class="img-fluid"
        src="calendar.svg" alt="Daily rides">
    </div>
  </div>
</div>
```

`col-md-4` + `col-md-8`
4/12 + 8/12 from **768px**.

800px: a 1:2 row



360px: stacked



`img-fluid`: fit the image.

Open the complete example ↗

Responsive layouts adapt to viewport width

A **responsive layout** adapts to the width of the browser's visible page area (the **viewport**). A **breakpoint** is a width threshold at which a layout rule changes.

Available breakpoints

Bootstrap includes six default breakpoints, sometimes referred to as *grid tiers*, for building responsively. These breakpoints can be customized if you're using our source Sass files.

Breakpoint	Class infix	Dimensions
X-Small	*None*	<576px
Small	`sm`	≥576px
Medium	`md`	≥768px
Large	`lg`	≥992px
Extra large	`xl`	≥1200px
Extra extra large	`xxl`	≥1400px

Source: Bootstrap · Breakpoints

For our Citi Bike layout, choose **Medium (`md`): 768px**.

- **768px and wider:** text and calendar share a row in a 1:2 ratio.
- **Below 768px:** text appears above the calendar.

The content stays the same; its arrangement changes.

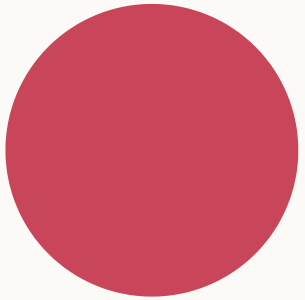
SVG describes graphics with elements

Scalable Vector Graphics (SVG) is a markup language for describing two-dimensional graphics with elements, attributes, and coordinates.

Each element describes a shape or label in the drawing.



<rect>



<circle>

Source: MDN · SVG

Draw one rectangle inside an SVG

The `svg` element contains the drawing. A `rect` element describes a rectangle.

```
<svg width="240" height="180">
  <rect x="20" y="20"
    width="100" height="80" />
</svg>
```

`width` and `height` set the rectangle’s size.



Drawing area: 240 × 180

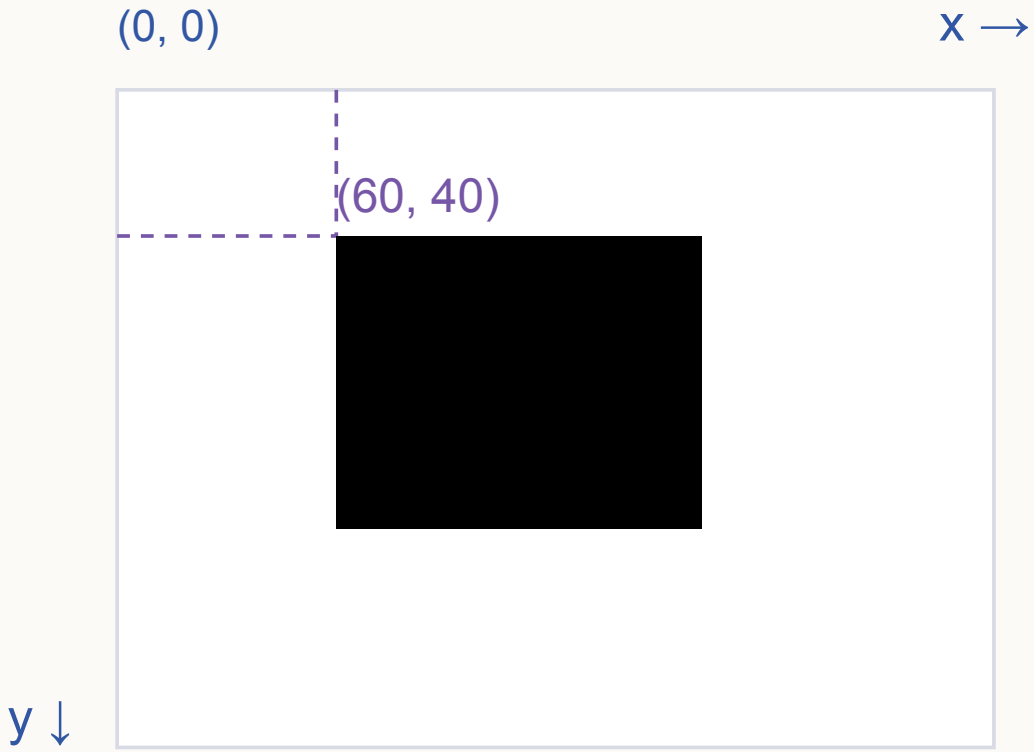
Position the rectangle with x and y

`x` is the horizontal position. `y` is the vertical position of the rectangle's top-left corner.

```
<rect x="60" y="40"
  width="100" height="80" />
```

x increases rightward.

y increases downward.

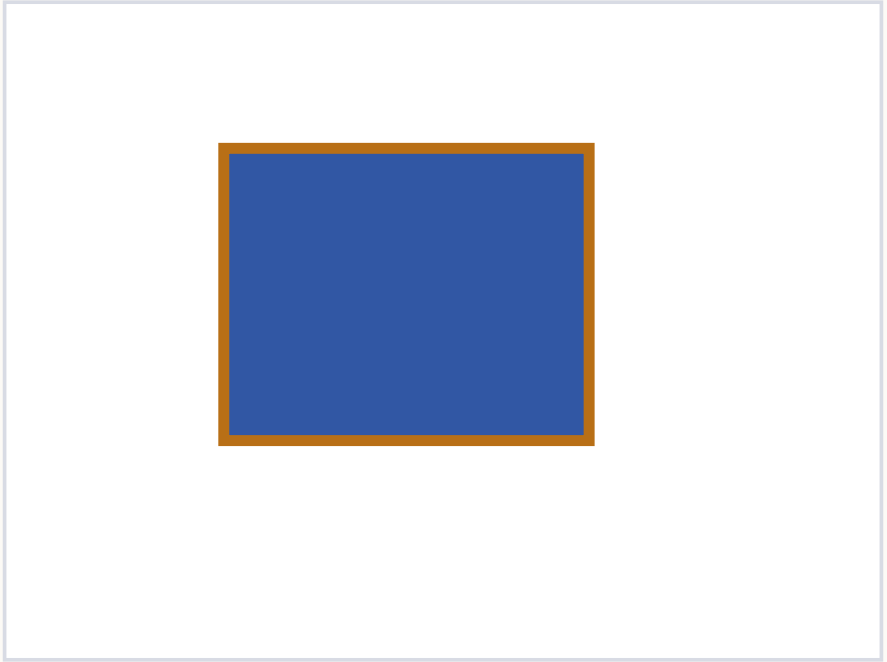


Use fill and stroke for different jobs

Fill colors the shape’s interior. **Stroke** draws its outline.

```
<rect x="60" y="40"
  width="100" height="80"
  fill="#3157a4"
  stroke="#b86f16"
  stroke-width="3" />
```

`stroke-width` sets the outline’s thickness.

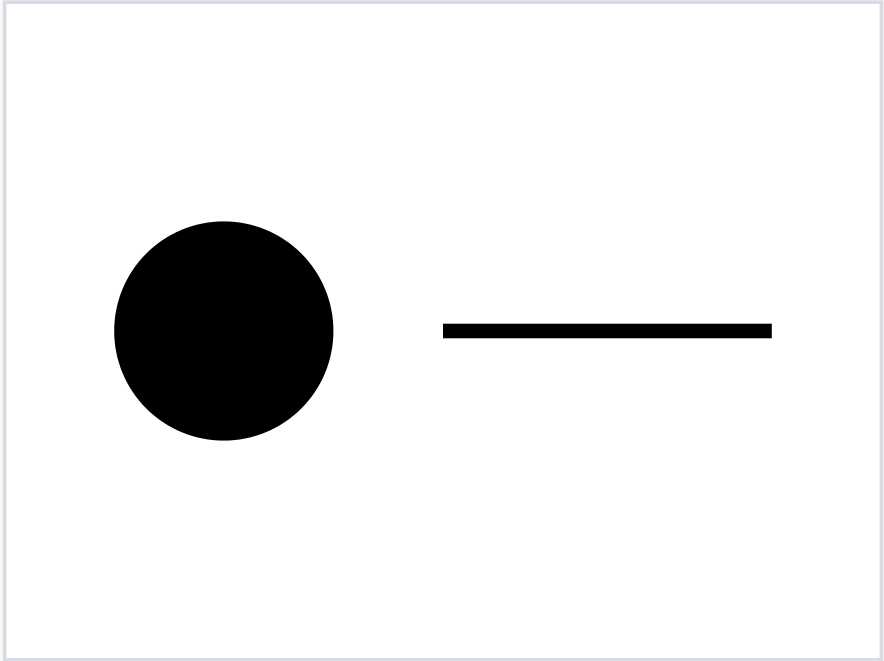


Drawing area: 240 × 180

Use circle and line for other marks

A `circle` has a center and radius. A `line` connects two endpoints.

```
<circle cx="60" cy="90"
  r="30" />
<line x1="120" y1="90"
  x2="210" y2="90"
  stroke="black"
  stroke-width="4" />
```



Drawing area: 240 × 180

Add text to the calendar cell

A `text` element places a label in the drawing.

```
<text x="70" y="65">4</text>
<text x="70" y="100">538</text>
```

The rectangle stays in place. Add the date and ride count.

Text style: white · 24px Arial.
April 4: **538 rides**.



Drawing area: 240 × 180

Group the shape and its labels

A `g` element groups related SVG elements. A translation moves the group together.

```
<g transform="translate(60,40)">
  <rect width="100" height="80"
    fill="#3157a4" stroke="#b86f16"
    stroke-width="3" />
  <text x="10" y="25">4</text>
  <text x="10" y="60">538</text>
</g>
```

Child coordinates are now relative to the group.



Drawing area: 240 × 180

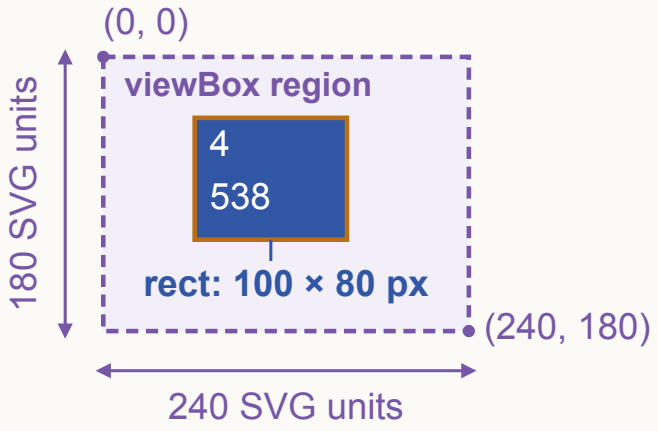
Resize the graphic without changing its coordinates

The `viewBox` specifies the **internal coordinate region**; `width` and `height` set the **display size**. Both rectangles stay **100 × 80 SVG units**; the browser scales them with the whole SVG.

```
<svg width="240" height="180"
  viewBox="0 0 240 180">
```

SVG display: 240 × 180 CSS px

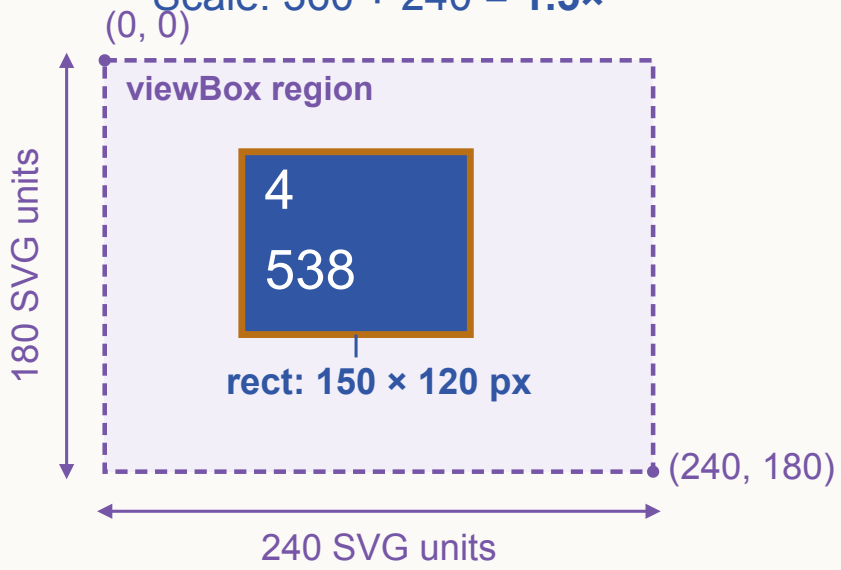
Scale: 240 ÷ 240 = **1×**




```
<svg width="360" height="270"
  viewBox="0 0 240 180">
```

SVG display: 360 × 270 CSS px

Scale: 360 ÷ 240 = **1.5×**



A calendar repeats the same SVG cell

```
<svg xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 240 100"
  font-family="Arial" font-size="24">
  <g transform="translate(10,10)">
    <rect width="100" height="80"
      fill="#87aed4" />
    <text x="10" y="25">1</text>
    <text x="10" y="60">299</text>
  </g>
  <g transform="translate(130,10)">
    <rect width="100" height="80"
      fill="#a3c1e1" />
    <text x="10" y="25">2</text>
    <text x="10" y="60">219</text>
  </g>
</svg>
```



- Repeat one `g` for each date.
- Change `translate` for its row and column.
- Update the date, count, and fill.

30 dates form the month.
View the full April calendar ↗

Explain the page in three layers



HTML

Headings, paragraphs, and sections.

CSS

Color, spacing, and the two-column layout.

SVG

Shapes, positions, and labels in the calendar.

Your turn: choose one visible change. Name the layer and describe the result you expect.

Open the finished page · Next: JavaScript and D3.